

Lenguajes de programación

¿Qué es un lenguaje de programación?

La programación es el proceso de escribir, probar, depurar, compilar y mantener el código fuente de un programa informático. Todos los lenguajes de programación se forman a partir de un conjunto de símbolos y una serie de reglas léxico/morfologías, que componen todas las estructuras válidas en el lenguaje y su significado.

Un **lenguaje de programación** es un sistema formal que permite escribir instrucciones que un ordenador puede interpretar y ejecutar.

Tipos de lenguajes

Los lenguajes se pueden clasificar según su nivel de abstracción:

Lenguajes de bajo nivel

- Cercanos al hardware
- Difíciles de leer
- Ejemplo: ensamblador

Lenguajes de alto nivel

- Más fáciles de aprender y usar
- Independientes del hardware
- Ejemplos: Python, Java, C++

Datos y expresiones

Datos

Se entiende **dato** como una expresión general que describe los objetos con los que operan los ordenadores.

Los datos pueden ser **primitivos** o **compuestos**, además, cualquier tipo de dato se puede establecer como constante, no varían durante la ejecución del programa, o variable, su contenido puede modificarse.

Los datos primitivos no se pueden descomponer en otros más simples, son:

- Datos numéricos
- Caracteres
- Booleanos

Los datos compuestos contienen otros datos, pueden ser de tipo estático, es decir, siempre ocupan el mismo tamaño en memoria:

- Arrays
- Registros

O por otro lado pueden ser dinámicos, es decir, pueden cambiar su tamaño en memoria durante la ejecución:

- Listas
- Pilas
- Colas
- Diccionarios

Expresiones

Llamamos expresión al conjunto de operador y operandos. Los operadores se pueden agrupar en 4 categorías:

- Aritméticos: +, -, *, /, MOD, DIV, ^...
- Alfanuméricos: + (concatenación)...
- Relacionales: =, >, <, >=, <=, <>, ==...
- Lógicos: NOT, AND, OR...

Estructuras básicas

Las estructuras de control determinan el flujo de ejecución de un programa.

Condicionales

Las estructuras condicionales nos permiten ejecutar diferentes conjuntos de instrucciones en función de la condición o condiciones que seleccionemos.

Podemos utilizar estructuras if para realizar comprobaciones o estructuras switch..case para elegir un curso de acción en función de un valor.



Resumen

Permiten tomar decisiones

```
int edad = 18;

if (edad >= 18) {
    System.out.println("Mayor de edad");
} else {
    System.out.println("Menor de edad");
}
```



Bucles

Las estructuras cíclicas (bucles) nos permiten ejecutar el mismo set de instrucciones un número determinado o indeterminado de veces, por ejemplo, las estructuras for se emplean cuando conocemos cuantas veces queremos repetir las instrucciones o queremos recorrer todos los elementos de un dato compuesto.

Por otro lado, las estructuras while se emplean cuando no sabemos el número de repeticiones, pero queremos que se sigan ejecutando hasta que se cumpla una condición. Existen variaciones como do while, que permite ejecutar el while al menos una vez o las estructuras.



Resumen

Permiten repetir instrucciones.

Bucle for

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```



Bucle while

```
int i = 0;

while (i < 5) {
    System.out.println(i);
    i++;
}
```



Funciones y procedimientos

Las estructuras de funciones y procedimientos nos permiten ejecutar sets de instrucciones concretos, esto ayuda a reducir la cantidad de líneas repetidas en los programas.

Resumen

Permiten reutilizar código.

```
public static int sumar(int a, int b) {  
    return a + b;  
}
```



Uso:

```
int resultado = sumar(3, 4);
```



Las funciones devuelven algun tipo de valor como resultado, por otro lado, los procedimientos solo ejecutan acciones. A nivel interno este tipo de estructuras se consideran variables con líneas de código en su "interior".

Comentarios

Los comentarios cumplen una función indispensable en cualquier lenguaje de programación, su función es ser ignorados por la ejecución del programa, lo que nos permite usarlos para documentar o explicar el código. Para que un comentario se identifique como tal debe ir acompañado de un símbolo especial, en Java es //.

Resumen

Sirven para documentar el código.

```
// Esto es un comentario de una línea  
  
/*  
    Esto es un comentario  
    de varias líneas  
*/
```



Sintaxis y semántica

Sintaxis

Es el conjunto de reglas que determinan cómo se escriben correctamente las instrucciones.

✓ Ejemplo

Esto es correcto:

```
int x = 5;
```



✗ Ejemplo

Esto es incorrecto:

```
int x = ;
```



Semántica

Es el significado de las instrucciones.

🔧 Ejemplo

- Sintácticamente correcto
- Semánticamente incorrecto (error en ejecución)

```
int x = 5 / 0;
```

